Assignment 2

第二次实践作业：FlashAttention-2

参考 Stanford CS336 Spring 2026 Assignment 2（Systems）

- 官方仓库：https://github.com/stanford-cs336/assignment2-systems <https://github.com/stanford-cs336/assignment2-systems>
- 任务详情：https://github.com/stanford-cs336/assignment2-systems/blob/main/cs336_assignment2_systems.pdf <https://github.com/stanford-cs336/assignment2-systems/blob/main/cs336_assignment2_systems.pdf>
- Assignment 分数：15 分
- 作业类型：个人作业
- 评测方式：统一评测脚本

一、任务

实现 FlashAttention-2：

- Forward：必须使用 Triton 实现。
- Backward：必须使用 Triton 实现，不允许使用 PyTorch 常规计算代替。
- 本课程统一使用 `causal=False`（非因果注意力）。
- 使用分块计算和数值稳定的 Softmax，不得显式构造完整的 $(N\times N)$ 注意力矩阵。
- 具体算法与实现方式参考 CS336 官方 Assignment 2 中 FlashAttention 相关内容。

代码框架：

```python
import torch

class FlashAttention2(torch.autograd.Function):
    @staticmethod
    def forward(ctx, q, k, v):
        # Triton forward
        raise NotImplementedError

    @staticmethod
    def backward(ctx, grad_out):
        # Triton backward
        return grad_q, grad_k, grad_v
```

统一使用 `FlashAttention2.apply(q, k, v)`，其中 `q/k/v` 的形状为 `[n_heads, sequence_length, d_head]`。（不额外传入 causal 参数）

1800字

二、开发机环境

本次实验使用与第一次实践作业相同的公共 Python 环境，已配置 PyTorch 2.11.0+cu128、CUDA 12.8 和 Triton 3.6.0，GPU 为 NVIDIA A100 80GB。

登录开发机后执行：

```
source /lmfs2026/env/a1/bin/activate
```

检查环境：

Plain Text | 复制代码

```
python -c "import torch, triton; print(torch.__version__, torch.version.cuda,
```

本次实验无需额外安装依赖，不需要手动配置 `CUDA_HOME` 或 CUDA 相关环境变量。

注意：公共环境为多人共享，不得自行安装、升级或修改公共环境中的依赖。

2. 实验目录

官方作业代码及测试数据已统一存放在开发机系统盘：

- **官方代码：** `/lmfs2026/data/assignment2-systems`
- **测评数据：** `/lmfs2026/data/2-fa2-data`

学生代码保存在个人目录：

```
mkdir -p /lmfs2026/$USER/assignment2
cd /lmfs2026/$USER/assignment2
```

公共代码和测试数据仅供读取，不得修改或删除。学生可参考官方代码框架，在个人目录中完成实验。

三、评测

本次实验通过统一评测脚本进行数值正确性与运行效率评估。

1. 数值正确性

评测脚本比较学生实现与参考实现的前向输出 `o`，以及反向梯度 `grad_q`、`grad_k`、`grad_v`。

本课程统一使用 `causal=False`，反向传播的测试损失为：

```
loss = o.sum()
loss.backward()
```

测试数据存放于 `/lmfs2026/data/2-fa2-data`：

1800字

测试文件	n_heads	sequence_length	
`fa2_nheads16_seq4128_dhead64.npz`	16	4128	
`fa2_nheads32_seq8576_dhead128.npz`	32	8576	

输入为 `float32`，每份数据包含 `q`、`k`、`v`、`o`、`grad_q`、`grad_k`、`grad_v`。

数值比较采用：

```
torch.allclose(actual, expected, atol=eps, rtol=0)
```

精度系数：

精度要求	
`atol=1e-3` 全部通过	
`atol=1e-2` 全部通过	
`atol=1e-1` 全部通过	
以上均未通过	

以前向输出和三个反向梯度在全部测试配置中的最差精度等级为准。

2. 运行效率

使用统一评测脚本测试 Forward + Backward 的运行时间。

/ai/lmfs2026/data/evaluate_assignment2.py Python | 运行代码 复制代码

使用 Triton Benchmark 进行预热与重复计时，正式评测统一运行参数，记录稳定运行时间，不计入首次编译时间。

注意：

- 学生实现必须能够在单张 A100 80GB GPU 上完成规定的前向、反向及性能测试。
- 若正式评测过程中因学生实现的显存需求超过 80GB 而发生 OOM（Out of Memory），本次实验直接判定为 Fail，最终得分为 0。

根据全班正常结束测评同学的运行时间排名分配效率分：

最终得分 = 效率分 × 数值精度系数（满分 15 分）

四、提交

学生在目录 `/ai/lmfs2026/data/assignment2` 中建立个人命名的文件夹（如 /ai/lmfs2026/data/assignment2/zhangsan），在文件夹下保存：

1. `flash_attention.py`：包含 `FlashAttention2` 类及 Triton Forward、Backward 实现。
2. 其他必要的自编源码文件。
3. 简要的运行说明及优化设计。

要求：

- 使用统一接口 `FlashAttention2.apply(q, k, v)`。
- 不得使用现成 FlashAttention、PyTorch SDPA 或其他完整注意力算子替代核心实现。
- 不得修改公共测试数据及评测脚本。